



Combinational Logic

Made By Mahmoud Ismail & Moheb Mikhael

2024/2025



Agenda

- Revision (Building code & code outlines).
- Lab 1.
- Behavioural modelling (always block).
- If & case statements.
- Lab 2.
- Gate Level & structural modelling.
- Lab 3.
- Recap and Compare between styles.

Building code & code outlines

Verilog HDL basics revision

- HDL stands for Hardware Description Language.
- Think as **Hardware** not **Programming**.
- Case sensitive, **Out** not equivalent to **out**.
- Lines ends with ;
- Comments like C++, // for one line, /* .. */ for a block.
- The code is executed in parallel.
- Don't start with coding !!
Analyse your design first.

Building code revision

- The file should be extended with .v
- The main build of a module:

```
1 module Name_of_the_module (  
2     input A,B, input [7:0] C,  
3     output y, output [3:0] out  
4 );  
5  
6 //the main code  
7  
8 endmodule
```

OR

```
1 module Name_of_the_module ( A, B, C, y, out );  
2  
3 input A,B;  
4 input [7:0] C;  
5 output y;  
6 output [3:0] out;  
7  
8  
9 //the main code  
10  
11 endmodule
```

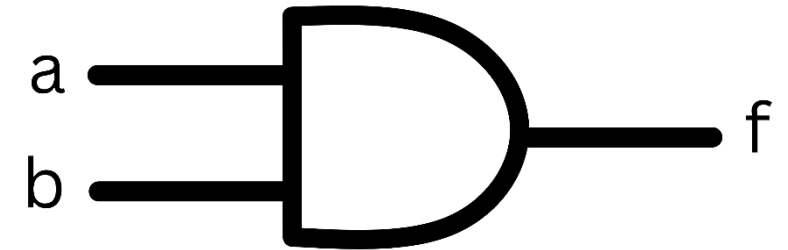
Preferred one

Assignment revision

assign means that, whenever any input signal changes, the output signal will be changed.

- It describes the behaviour of the combinational logic.

```
1 module assignment_example (input a,b, output f);  
2  
3 assign f = a&b;  
4  
5 endmodule
```



Assignment revision

`wire` is used to make an internal wire.



```
1  module Assignment_example (input a,b,c, output f);  
2  
3  wire w1;  
4  
5  assign w1 = a^b;  
6  assign f = w1&c;  
7  
8  endmodule
```

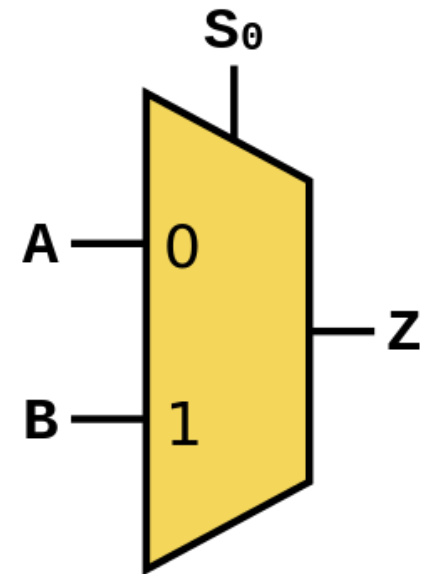
Assignment revision

Remember the conditional operator.

- condition? true : false



```
1 module MUX_2to1 (input a,b,sel, output out);  
2  
3 assign out = sel? b : a;  
4  
5 endmodule
```

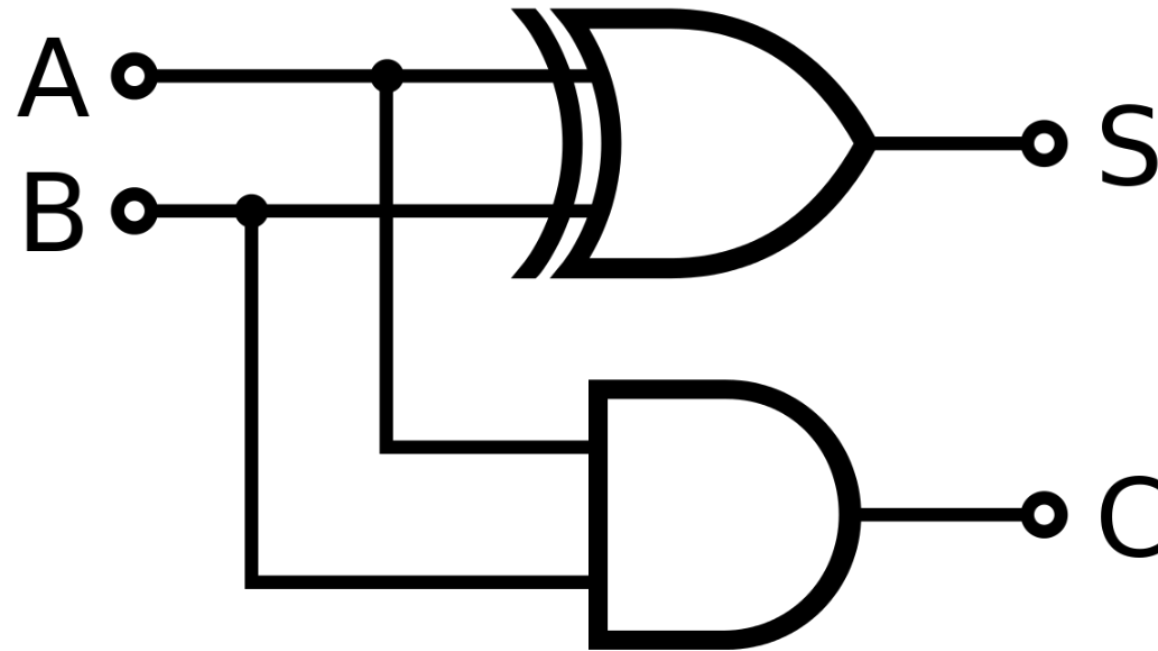


Lab 1

Lab 1

Implement the following design.

- **Bonus:** implement it using one assignment statement.



Lab 1 solution



```
1  module Lab1_2 (input a,b,  
    output s,c);  
2  
3  assign s = a^b;  
4  assign c = a&b;  
5  
6  endmodule
```



```
1  module Lab1_1 (input a,b,  
    output s,c);  
2  
3  assign {c , s} = a+b;  
4  
5  endmodule
```

What is the Behavioural modelling?

Behavioural modelling

- Describe the behaviour of the circuit, what the circuit does and how.
- It's a high-level approach to describing digital circuits.

```
1 module Gate_level_modeling (input a,b,  
  output s,c);  
2  
3   assign s = a^b;  
4   assign c = a&b;  
5  
6   endmodule
```

```
1 module behavioural_modeling (input a,b,  
  output s,c);  
2  
3   assign {c , s} = a+b;  
4  
5   endmodule
```

Procedural block

- `always` and `initial` blocks.
- Statements inside a procedural block are executed sequentially, line-by-line.
- Provide powerful ways of doing complex designs.
- Today, we will talk about `always` block.
- It supports C-like control structure such as `if`, `for`, `while`, `case`.
- Can be used for combinational and sequential logic.
- Remember the behavioural model.

Always block build

```
1  module Always_block_build (input a, b, output reg s);  
2  
3  reg w1;  
4  
5  always @ (a or b)  
6  begin //always  
7  
8  w1 = a^b;  
9  s = w1&b;  
10  
11 end //always  
12  
13 endmodule
```

Always block build (output reg / reg)

- In Verilog **wire** and **reg** are two fundamental data types used to represent signals.
- **wire** is a direct connection between two points, doesn't store a value on its own.
- **reg** can store a value, It remembers the value assigned to it until it's changed.
- always block contains procedural assignments, which are executed sequentially.
- **Rule:** use **output reg** instead of **output** – use **reg** instead of **wire**

Always block build (main build)

- `always @ (a or b)`: the head of the block.
- `(a or b)`: called sensitivity list, means if a or b changes, get into the always block.

Can be written as `(*)`, that means if any input signal changes get into the block.

- `begin/end`: boundaries of the block, like curly brackets in C++.

If & Case Statements

If statement

- Only for `always` and `initial` blocks.
- `begin/end`: boundaries of the block.

• Note:

If `((a == 0) || (b && enable))`

As C++.

```
1 module If_statement_example (input enable,a,b, output reg [1:0] s);
2
3 always @ (*)
4 begin //always
5
6
7 if (enable && a)
8     begin //if
9         s = 2'b11;
10    end //if
11
12 else if (enable && b)
13     begin //else if
14         s = 2'b10;
15     end //else if
16
17 else
18     begin //else
19         s = 2'b00;
20     end //else
21
22
23 end //always
24
25 endmodule
```

Case statement

- Only for `always` and `initial` blocks.
- No `begin`, and its end is `endcase`.

```
1 module Case_statement_example (input [1:0] opcode,a,b, output reg [1:0] s);
2
3 always @ (*)
4 begin
5
6 case (opcode)
7
8 2'b00:
9 begin
10 s = a+b;
11 end
12
13 2'b01:
14 begin
15 s = a&b;
16 end
17
18 2'b10:
19 begin
20 s = a|b;
21 end
22
23 default:
24 begin
25 s = ^a;
26 end
27
28 endcase
29
30 end
31
32 endmodule
```

Important notes for If and Case

- Each output must be assigned for each **case**.
- **If** statement must be ended with **else**.
- **Case** statement must contain all possible signals.

Why?

- To avoid unintentional latches, it will be covered later.
- **Rule**: avoid unintentional latches.

Lab 2

Lab 2

- Implement an ALU with these specifications:

Op-code	Operation
2'b00	$A + B$ (add)
2'b01	$A - B$ (subtract)
2'b10	$A \& B$ (AND gate)
2'b11	Pass A

Signal	Size (bit)
enable	1
A	4
B	4
out	4
opcode	2

Bonus: add carry_flag signal.

Hint: add it for arithmetic operation.

Think as a designer

Lab 2 solution

```
1 module Lab2_ALU (input [3:0] A,B, input enable, input [1:0] opcode,  
2 output reg [3:0] out , output reg cout);  
3  
4 always @ (*)  
5 begin //always  
6  
7 if (enable)  
8 begin //if  
9  
10 case (opcode)  
11  
12 2'b00:  
13 begin //2'b00  
14 {cout , out} = A + B;  
15 end //2'b00  
16  
17 2'b01:  
18 begin //2'b01  
19 {cout , out} = A - B;  
20 end //2'b01
```

```
21  
22 2'b10:  
23 begin //2'b10  
24 out = A & B;  
25 cout = 1'b0;  
26 end //2'b10  
27  
28 default:  
29 begin  
30 out = A;  
31 cout = 1'b0;  
32 end  
33  
34 endcase  
35  
36 end //if  
37  
38 else  
39 begin //else  
40 out = 4'b0;  
41 cout = 1'b0;  
42 end //else  
43  
44  
45 end //always  
46  
47 endmodule
```


Break



Part 2

▪ sneaky syntax errors



**In this section , we
will learn about:**

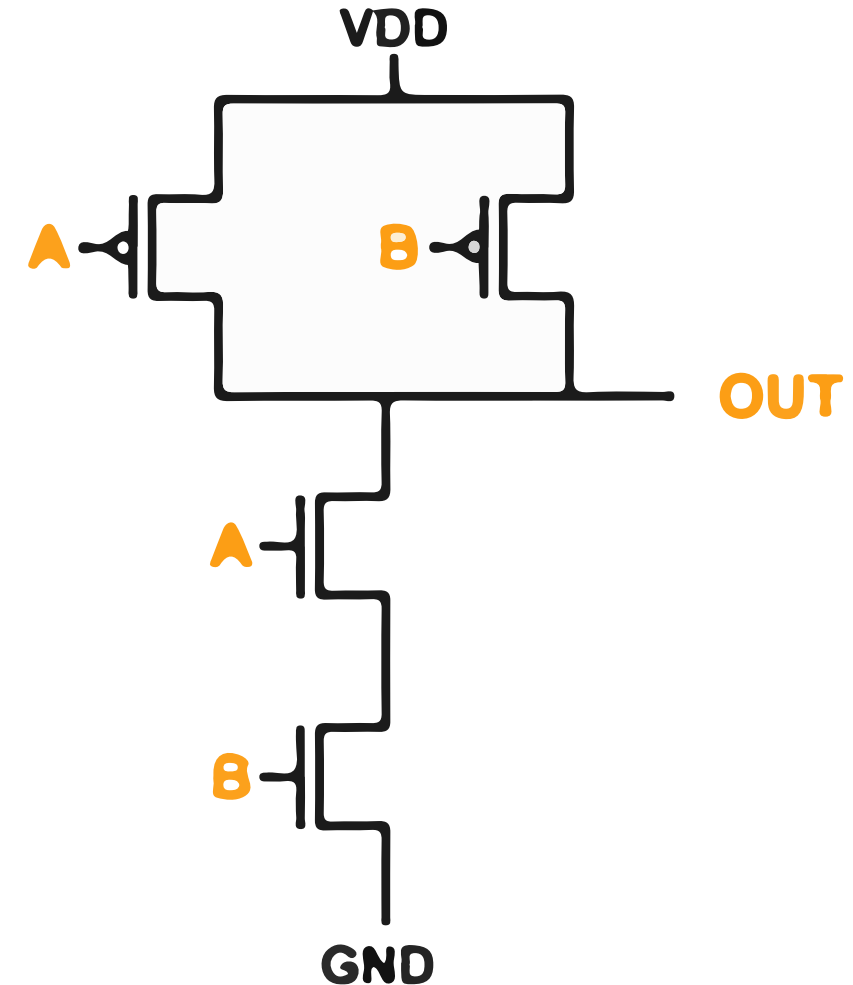
- **Gate-level modeling**
- **Structural modeling**
- **Dataflow level modeling**
- **Behavioral modeling**
- Switch (transistor) level (out of the scope)

- **Switch (transistor) level**

- **What does it mean?**

▪ Switch (transistor) level

- What does it mean?



**In this section , we
will learn about:**

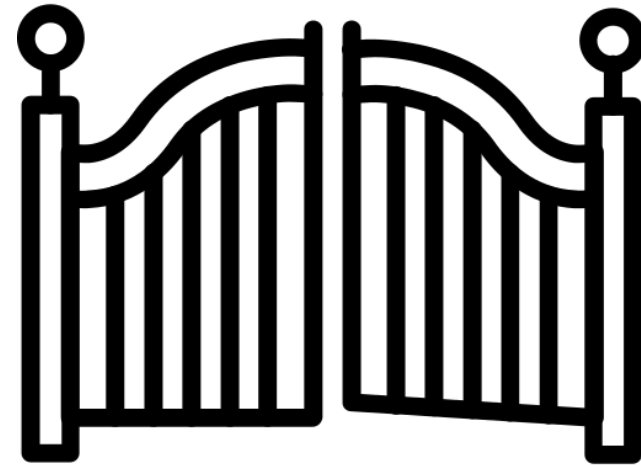
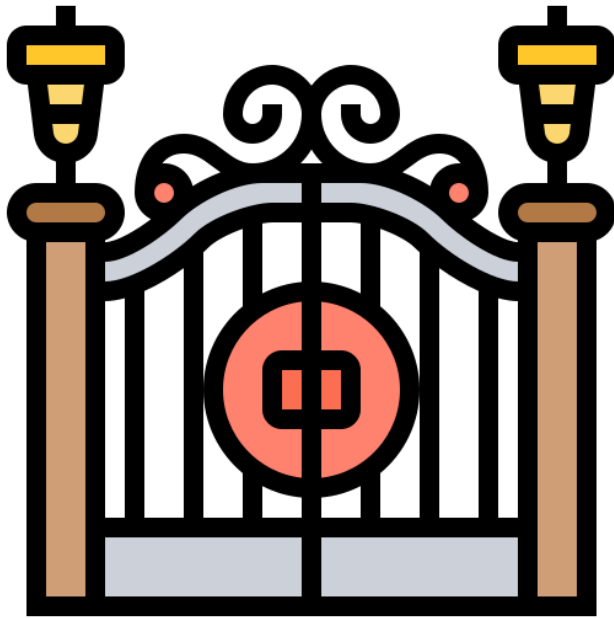
- **Gate-level modeling**
- **Structural modeling**
- **Dataflow level modeling**
- **Behavioral modeling**
- Switch (transistor) level (out of the scope)

- **Gate-level modeling**

- **What does it mean?**

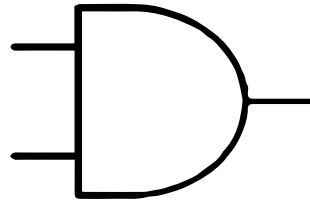
▪ Gate-level modeling

- What does it mean?

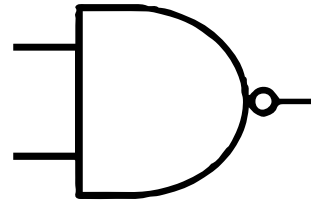


■ Gate-level modeling

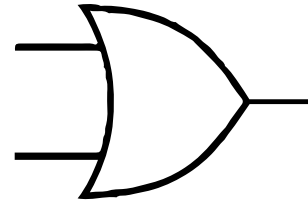
■ What does it mean?



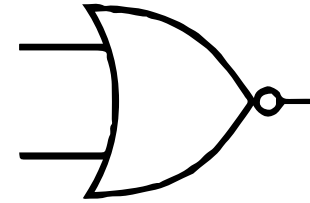
AND



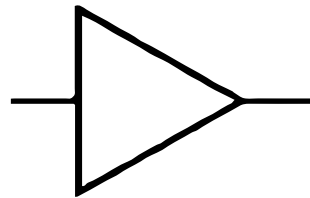
NAND



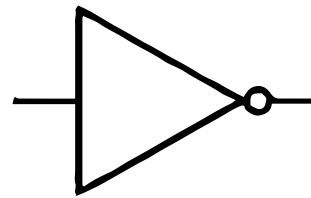
OR



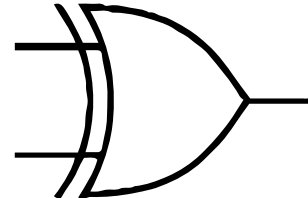
NOR



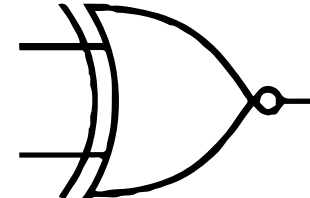
BUFFER



NOT



XOR



XNOR

▪ Gate-level modeling

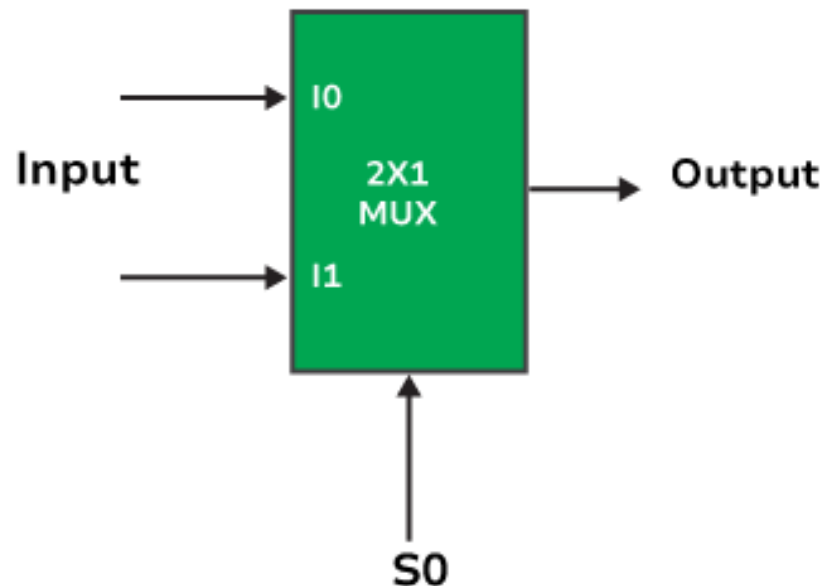
- **Gate-level modeling** represents digital circuits using **basic logic gates**
- It provides **the lowest level of abstraction, directly** describing how **logic gates are interconnected** to form a circuit.

Example

▪ Gate-level modeling



2:1 Multiplexer



Truth Table

S_0	I_0	I_1	Y
0	0	X	0
0	1	X	1
1	X	0	0
1	X	1	1

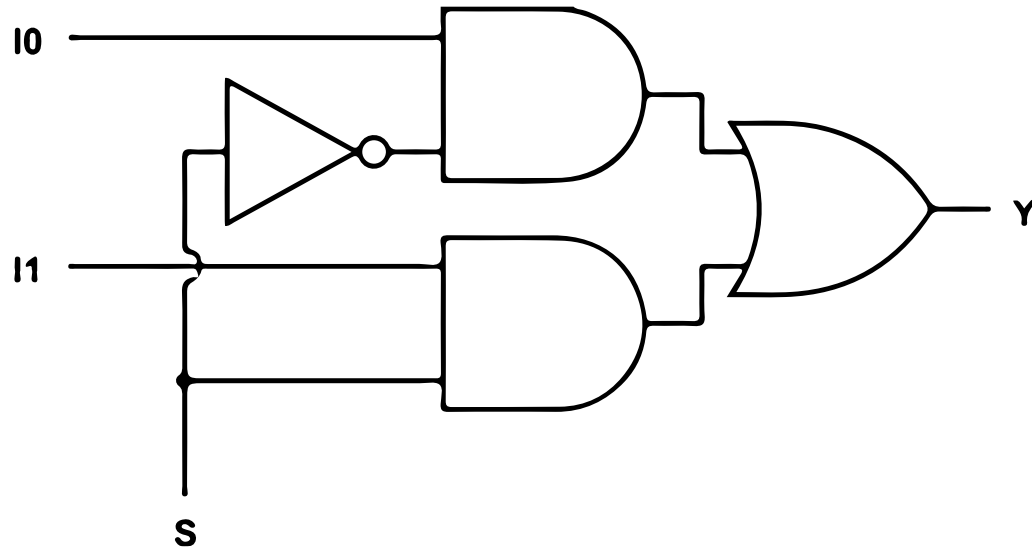


Example : MUX 2 to 1

▪ Gate-level modeling



2:1 Multiplexer



Truth Table

S_0	I_0	I_1	Y
0	0	X	0
0	1	X	1
1	X	0	0
1	X	1	1

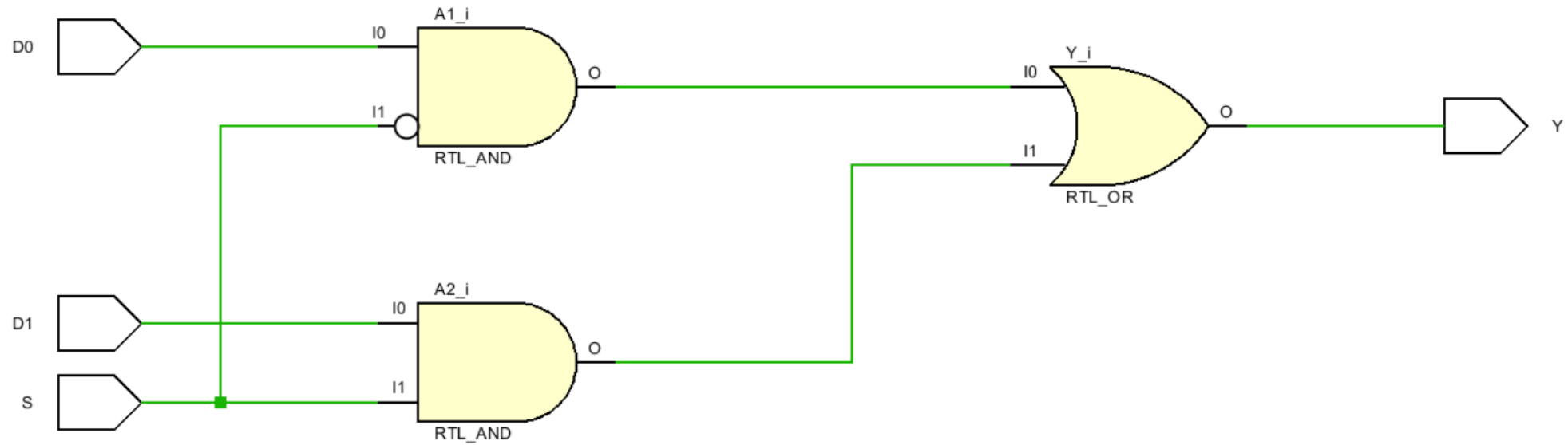


Example : MUX 2 to 1

▪ Gate-level modeling

```
module mux2to1 (output wire Y, input wire D0, D1, S);  
    wire S_not, A1, A2;  
  
        // S' = NOT S  
    and (A1, D0, ~ S); // A1 = D0 AND S'  
    and (A2, D1, S);   // A2 = D1 AND S  
    or  (Y, A1, A2);   // Y = A1 OR A2  
  
endmodule
```

▪ Gate-level modeling



**In this section , we
will learn about:**

- **Gate-level modeling**
- **Structural modeling**
- **Dataflow level modeling**
- **Behavioral modeling**
- **Switch (transistor) level (out of the scope)**

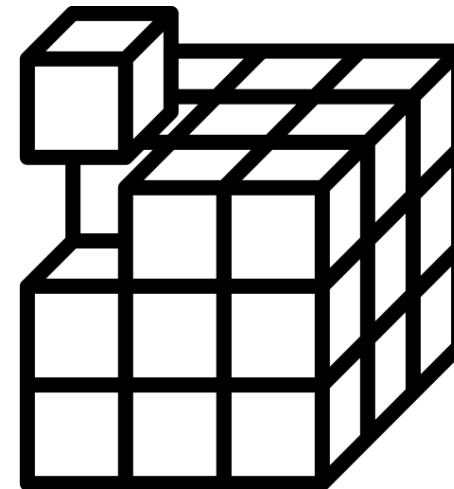
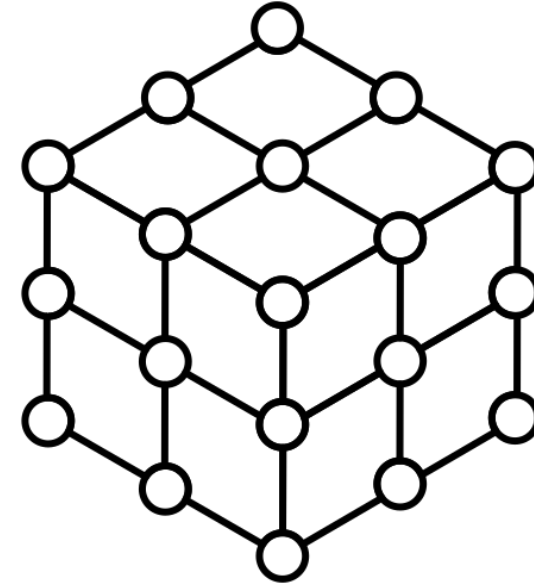
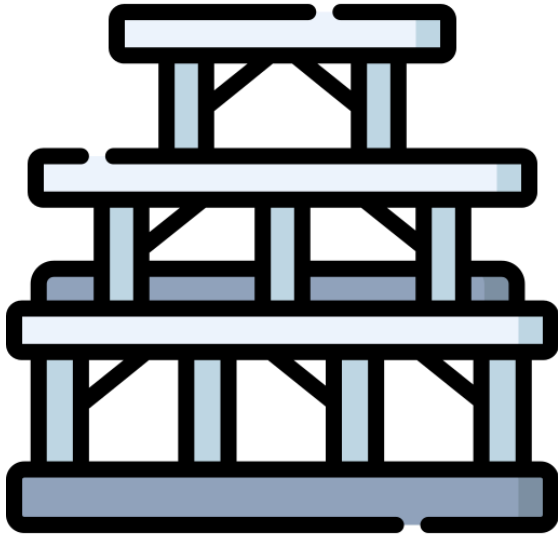
Structural modeling

▪ Structural modeling

▪ What does it mean?

■ Structural modeling

- What does it mean?



▪ Structural modeling

- **Structural modeling** connects **multiple modules** to build complex circuits using instantiations.
- **Instantiation** is the process of **linking** a **larger** module with the **smaller** modules **that compose it**.

```
Half_adder HF2(A2, S, D1, C1);
```

The order is important

Module Name → Half_adder (the module being instantiated)

Instance Name → HF2 (a unique identifier for this instance)

Port Connections → (A2, S, D1, C1) (signals connected to module ports) (inputs or outputs)

Example

▪ Structural modeling

```
module comparator_1bit(output eq, input A, B);  
    assign eq = (A * B) + ((1 - A) * (1 - B));  
  
endmodule
```

▪ Structural modeling

```
module comparator_2bit(output eq, input [1:0] A, B);  
    wire eq0, eq1;  
  
    // Instantiating two 1-bit comparators  
    comparator_1bit C0(eq0, A[0], B[0]);  
    comparator_1bit C1(eq1, A[1], B[1]);  
  
    // Final equality check using arithmetic  
    assign eq = eq0 * eq1; // Only equal if both bits are equal  
endmodule
```


■ Notes:

- Modules can be **separated** into **different files**, but **all** files must be compiled.
- Instantiation also can be :

```
module_name instance_name (.port_name1(signal1), .port_name2(signal2), ... );
```

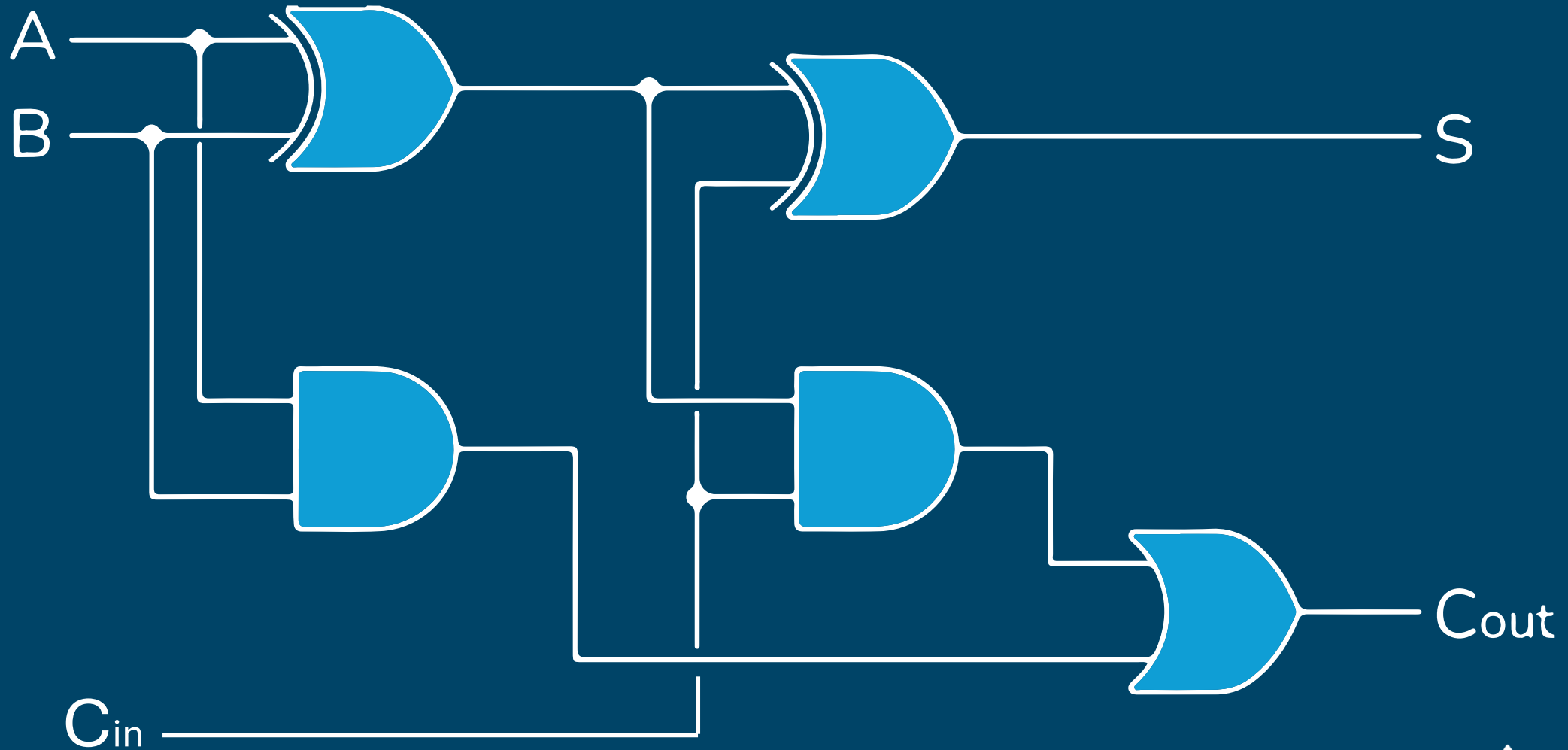
- **Signals can be reordered here**

▪ Structural modeling

- Also, **logic gates** can be used as in the previous slide
- Used modules can **be complex or simple**. In the previous example, simple assign statements were used for simplicity.

Example 2

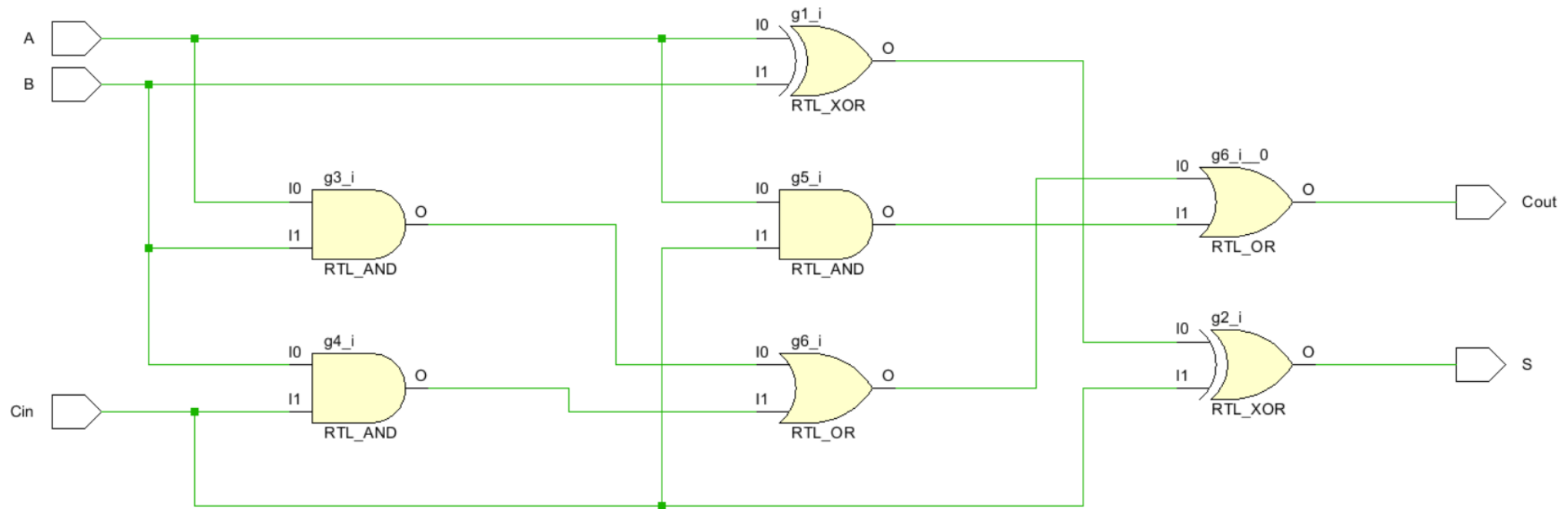
Example 2



▪ Example 2

```
module full_adder(output S, Cout, input A, B, Cin);  
    wire X1, A1, A2, A3;  
  
    xor (X1, A, B);  
    xor (S, X1, Cin);  
    and (A1, A, B);  
    and (A2, B, Cin);  
    and (A3, A, Cin);  
    or  (Cout, A1, A2, A3);  
endmodule
```

Gate-level modeling



- Example 2

- Is it hard?

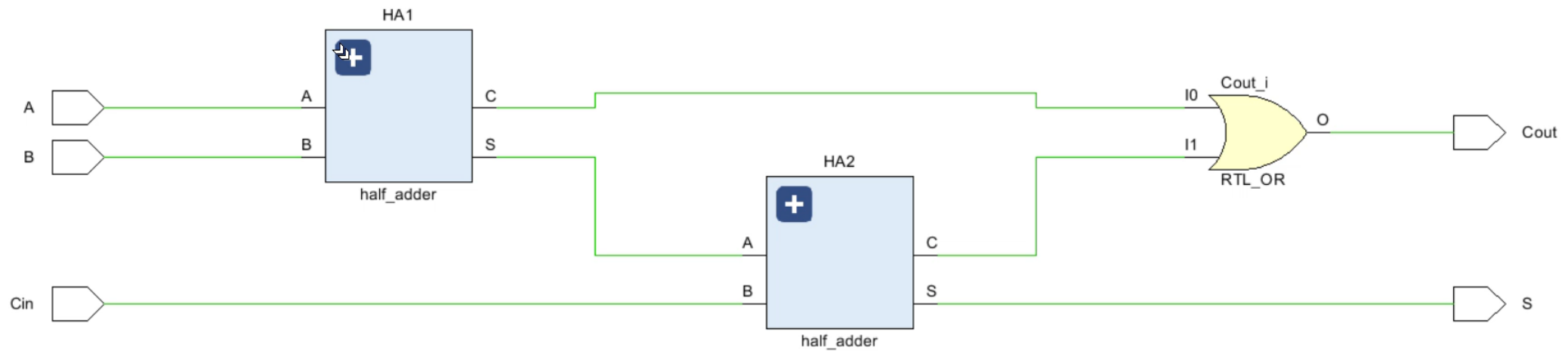
▪ Example 2

```
module half_adder(output S, C, input A, B);  
    xor (S, A, B); // Sum =  $A \oplus B$   
    and (C, A, B); // Carry =  $A \cdot B$   
endmodule
```

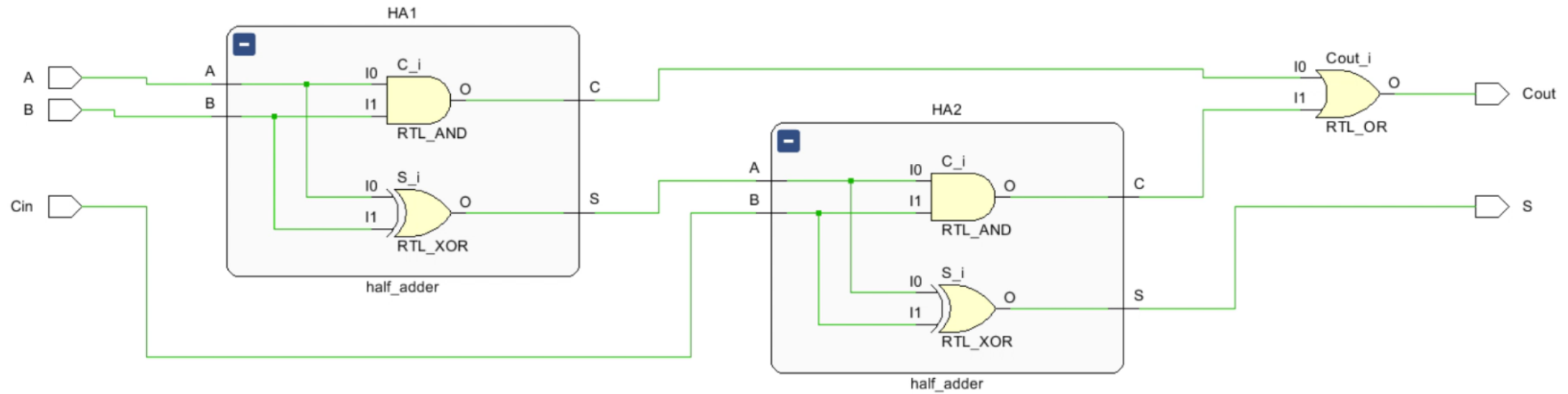

▪ Example 2

```
module full_adder(output S, Cout, input A, B, Cin);  
    wire S1, C1, C2;  
  
    // First Half Adder  
    half_adder HA1(S1, C1, A, B);  
  
    // Second Half Adder  
    half_adder HA2(S, C2, S1, Cin);  
  
    // Final Carry Calculation  
    or (Cout, C1, C2);  
endmodule
```

▪ Structural modeling



■ Structural modeling



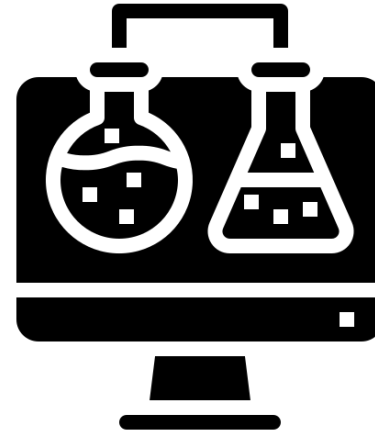
- **Example 2**

- **Still hard?**

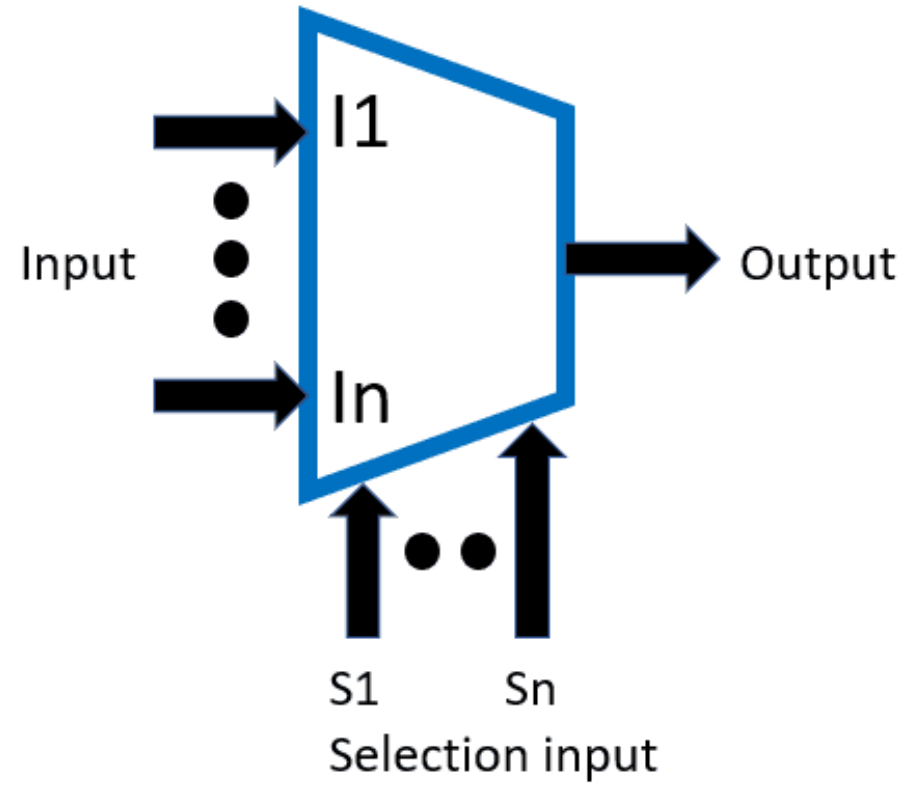
▪ What about this?

```
module full_adder ( input A, B, Cin, output Sum, Cout );  
    assign {Cout, Sum} = A + B + Cin;  
endmodule
```

LAB



LAB



■ LAB

- A **4-to-1 MUX** selects one of four inputs (D_0, D_1, D_2, D_3), where $D[3:0]$ based on two select lines (S_1, S_0), $S[1:0]$, and outputs the selected value.
- The truth table specifies which input is selected for each combination of the select lines.

S_1	S_0	Selected Input	Y (Output)
0	0	D_0	$Y = D_0$
0	1	D_1	$Y = D_1$
1	0	D_2	$Y = D_2$
1	1	D_3	$Y = D_3$

LAB_{solved}

```
module mux4to1 (  
    input wire [3:0] D,    // Data inputs  
    input wire [1:0] S,    // Select lines  
    output wire Y    );    // Output  
  
    // AND gates for selecting inputs based on select lines  
    wire Y0, Y1, Y2, Y3;  
    and (Y0, D[0], ~S[1], ~S[0]);  
    and (Y1, D[1], ~S[1], S[0]);  
    and (Y2, D[2], S[1], ~S[0]);  
    and (Y3, D[3], S[1], S[0]);  
  
    // OR gate to combine the selected inputs  
    or (Y, Y0, Y1, Y2, Y3);  
  
endmodule
```

- Use the (mux4to1) designed in the previous part to create an 8-to-1 multiplexer using instantiations.

■ LAB

- **Use the (mux4to1) designed in the previous part to create an 8-to-1 multiplexer using instantiations.**
- **Notice that now there are 8 input (data), [7:0] D, and 3 selection lines are needed [2:0] S .**

- Note: Selection lines are also inputs

- **Instantiation syntax:**

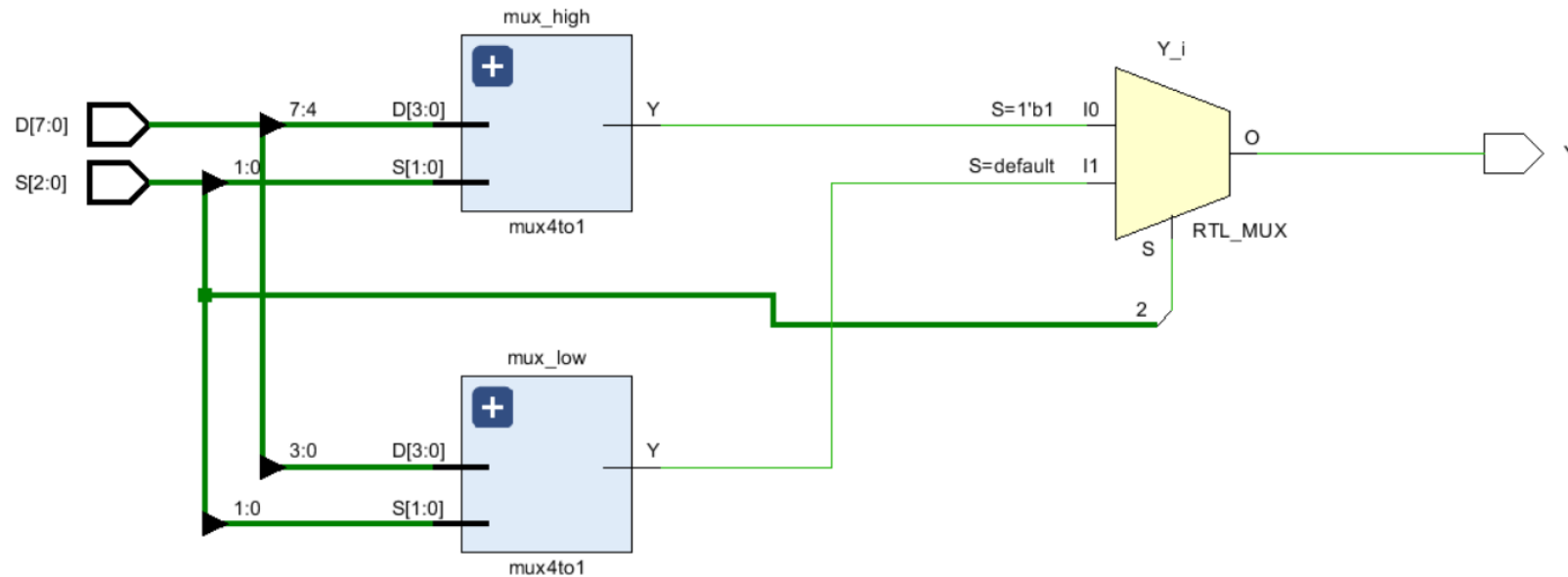
```
module_name instance_name (.port_name1(signal1), .port_name2(signal2), ... );
```

OR

```
module_name instance_name ( signal1, signal2 , ... );
```

LAB

- Use the (mux4to1) designed in the previous part to create an 8-to-1 multiplexer using instantiations.
- Hint: this will be its schematic.



LAB_{solved}

▪ Structural modeling

```
module mux8to1 ( input wire [7:0] D,    // Data inputs
                 input wire [2:0] S,    // Select lines
                 output wire Y );      // Output

    wire Y_low, Y_high;

    // Instantiate two 4-to-1 multiplexers
    mux4to1 mux_low (.D(D[3:0]), .S(S[1:0]), .Y(Y_low) );

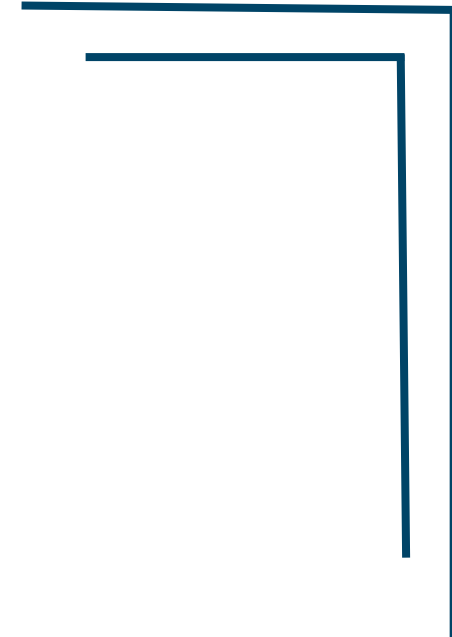
    mux4to1 mux_high (.D(D[7:4]), .S(S[1:0]), .Y(Y_high));

    assign Y = (S[2]) ? Y_high : Y_low;
    // 2-to-1 multiplexer to select between Y_low and Y_high based on S2

endmodule
```

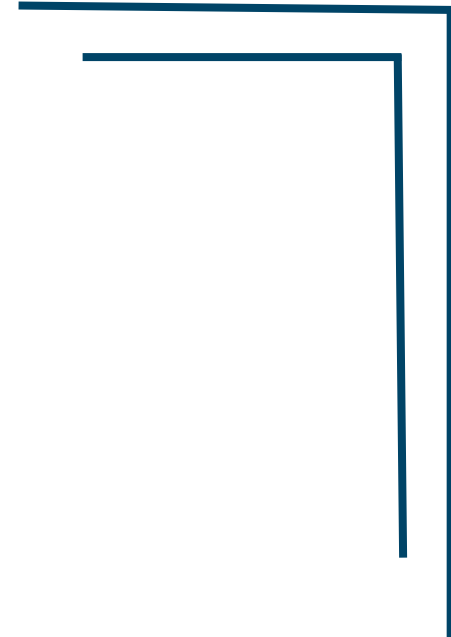






▪ Gate-level modeling

```
module mux2to1 (output wire Y, input wire D0, D1, S);  
    wire S_not, A1, A2;  
  
    not (S_not, S);           // S' = NOT S  
    and (A1, D0, S_not);      // A1 = D0 AND S'  
    and (A2, D1, S);          // A2 = D1 AND S  
    or (Y, A1, A2);           // Y = A1 OR A2  
  
endmodule
```



▪ Structural modeling

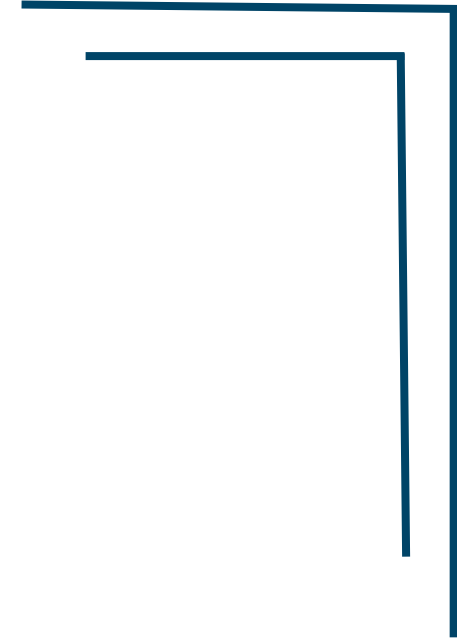
```
module and_gate(output wire Y, input wire A, B);  
    assign Y = A & B;  
endmodule
```

```
module or_gate(output wire Y, input wire A, B);  
    assign Y = A | B;  
endmodule
```

```
module not_gate(output wire Y, input wire A);  
    assign Y = ~A;  
endmodule
```

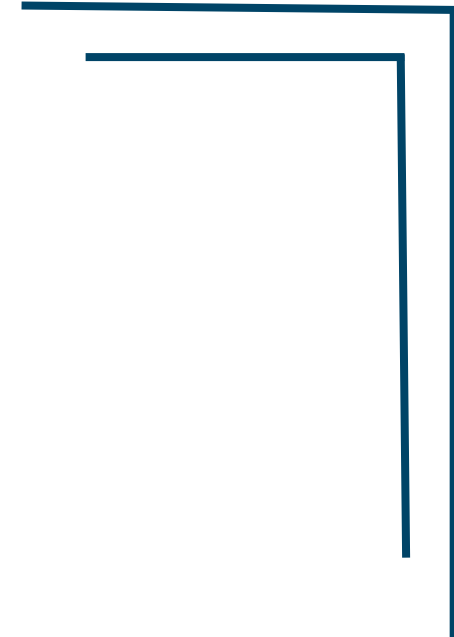
▪ Structural modeling

```
module mux2to1(output wire Y, input wire D0, D1, S);  
    wire S_not, A1, A2;  
  
    // Instantiate NOT gate  
    not_gate NOT1(S_not, S);  
  
    // Instantiate AND gates  
    and_gate AND1(A1, D0, S_not);  
    and_gate AND2(A2, D1, S);  
  
    // Instantiate OR gate  
    or_gate OR1(Y, A1, A2);  
endmodule
```



▪ Behavioral modelling

```
module mux2to1 ( output reg Y,  
    input wire D0, D1, S );  
  
    always @(*) begin  
  
        if (S)  
            Y = D1;  
        else  
            Y = D0;  
        end  
  
    endmodule
```

▪ Dataflow level modeling

```
module mux2to1 (  
    output wire Y,  
    input wire D0, D1, S  
);  
    assign Y = (S) ? D1 : D0;  
endmodule
```

■ comparison

Gate-Level

Uses basic logic gates
(`and` , `or` , `not`).

```
and (A1, D0,  
S_not);
```

✓ Low-level
control

✓ Good for
ASIC/FPGA

✗ Hard to debug

✗ Complex for large
circuits

Dataflow

Uses Boolean
equations and
`assign` .

```
assign Y = (S) ?  
D1 : D0;
```

✓ Compact and
readable

✓ Efficient for
combinational
logic

✗ Cannot handle
sequential logic

▪ comparison

Behavioral

Uses `always` blocks and control statements.

```
always @(*) begin
if (S) Y = D1; else
Y = D0; end
```

✓ Best for complex designs
✓ Supports sequential circuits

✗ Less control over hardware implementation

Structural

Uses module instantiation to connect components.

```
and_gate AND1 (A1,
D0, S_not);
```

✓ Good for hierarchical designs
✓ Reusability of components

✗ More verbose than dataflow and behavioral

In this section, we learned about:

- **Gate-level modeling**
- **Structural modeling**
- **Dataflow level modeling**
- **Behavioral modeling**
- Switch (transistor) level (out of the scope)

QUESTIONS



Thank you

